



Improving SIEM Rules Using Transformer-Based Rule Evasion Detection and Attribution

Erik Nordström¹(✉), Hannes Widéen¹, Valency Oscar Colaco²,
and Simin Nadjm-Tehrani²

¹ Sectra Communications AB, Linköping, Sweden
{erik.nordstrom,hannes.wideen}@sectra.com

² Linköping University, Linköping, Sweden
{valency.colaco,simin.nadjm-tehrani}@liu.se

Abstract. Enterprise networks are being increasingly targeted by sophisticated cyber threats, making the rapid detection of intrusions critical to limiting their impact. To this end, organisations deploy Security Information and Event Management (SIEM) systems, which monitor network and/or host activity to flag potential attacks using expert-written detection rules. In this context, one of the most widely adopted rule formats is SIGMA, which provides a standardised, publicly available specification for describing detection logic across different SIEM platforms. In this paper, we first analyse a representative subset of these SIGMA rules and show that a quarter of them can be easily evaded using straightforward techniques that modify commands syntactically while preserving their semantic meaning, causing traditional rule-matching approaches to fail. To address this limitation, we propose a transformer-based approach that captures the semantic relationship between commands and SIEM rules to identify potential evasion attempts. We then move on from detection to rule attribution, i.e., pinpointing which specific rule(s) an adversary attempted to bypass, thereby supporting attack diagnostics. Furthermore, our approach generates actionable recommendations to improve the evadable SIEM rules. Experimental evaluations show that our approach detects SIEM rule evasions with an accuracy of 71% and a false positive rate of 0.9%, while correctly attributing 98% of detected evasions to their corresponding SIEM rules. Performing an expert evaluation, security analysts, when presented with recommendations generated by our approach, reported that 31% could replace existing rules with minor adjustments, while 47% offered key insights for manual refinement. Finally, to demonstrate real-world impact, a subset of these recommendations were submitted to the SIGMA repository maintainers, resulting in several SIEM rules being fixed through our feedback.

Keywords: Security Information and Event Management · SIEM
Detection Rules · PowerShell · SIEM Rule Evasions · SIGMA
Detection Format

1 Introduction

In today’s digital landscape, enterprise networks face an ever-growing volume of cybersecurity threats that place sensitive data and critical operations at serious risk. High-profile incidents such as the 2020 SolarWinds breach, which compromised thousands of companies and government agencies [24], and the 2024 CrowdStrike outage, which crashed approximately 8.5 million systems in what has been widely described as the largest IT outage in history [25], underscore just how devastating these threats can be. To defend against such risks, organisations typically layer both active and passive security measures in a defence-in-depth strategy [31]. Active measures, such as firewalls and access control systems, work to prevent breaches from occurring, while passive measures help identify adversaries who have successfully breached their defences [5]. To implement such passive detection measures, organisations often utilise Security Information and Event Management (SIEM) systems, which collect and analyse network and/or host activity to identify suspicious behaviour and detect potential incidents. These systems primarily rely on *misuse detection*, also known as rule/signature-based detection, which flags events based on known malicious behaviour.

To discover potential attacks from these events, organisations predominantly rely on expert-written SIEM rulesets [1, 14, 18]. One widely adopted example is SIGMA [8], which provides a publicly available, standardised format for describing rule-based detection logic across different SIEM platforms. However, its public availability introduces a subtle vulnerability: adversaries can study the rules and evade detection by making slight modifications to commands while preserving their original malicious intent, thereby exploiting the blind spots in an organisation’s cyber defence plan. As a result, even rules that are effective at detecting known attacks can still be bypassed using evasive techniques [22].

Building on this observation, in this paper, we confirm that widespread SIEM rules from 2025 are indeed prone to evasions, which can often be achieved through straightforward syntactic modifications that preserve the command’s malicious

Table 1. SIEM rule evasion example

Category	Details
SIEM Rule	any where(process.command_line: ‘‘*nslookup*’’ and process.command_line: ‘‘*[1]*’’ and process.command_line: (‘‘*-q=txt http’’, ‘‘*-querytype=txt http*’’))
Affected Term	process.command_line: (‘‘*-q=txt http’’,
Used Technique	Substitution
Matching Command	(nslookup -q=txt https://some-site.com)[1]
Evading Command	(nslookup -type=txt https://some-site.com)[1]

intent. An example of such an evasion is illustrated in Table 1. In this example, simply substituting the `-q=txt` parameter with `-type=txt` allows the command to bypass SIEM detection. Such evasions are particularly concerning for platforms that are both widely deployed and actively targeted. One such platform is Windows PowerShell, a command-line shell and scripting language widely used in enterprise environments. The active targeting of PowerShell is further illustrated by Microsoft’s February 2026 Patch Tuesday, which addressed 58 vulnerabilities across Windows and related components, including six actively exploited zero-day flaws that enabled privilege escalation, code execution, and bypass of security features. Given the prevalence of PowerShell in enterprise environments and its frequent abuse in real-world attacks, ensuring that SIEM rules are robust against evasions in this domain is critical. For this reason, we focus specifically on the Windows/PowerShell subset of the SIGMA rules.

Since these evasion attempts modify commands syntactically while preserving their intended functionality (i.e., semantic meaning), traditional pattern-matching approaches often fail to detect them. To address this limitation, we propose a transformer-based model that captures the semantic relationship between commands and SIEM rules in order to detect potential evasions. Beyond detecting evasions, our approach also identifies which specific rule(s) an adversary attempted to bypass, enabling rule attribution to support attack diagnostics. Furthermore, our method produces actionable rule improvement recommendations to enhance the robustness of SIEM rules against evasion attempts. In this context, the contributions of this paper are summarised as follows:

- **Transformer-based SIEM Rule Evasion Detection:** We present a method for detecting SIEM rule evasions based on semantic similarity comparisons derived from transformer embeddings, enabling the identification of semantically equivalent but syntactically different commands.
- **SIEM Rule Attribution Under Evasions:** We introduce a rule-attribution mechanism that, for each incoming evasion attempt, identifies the most likely SIEM rule the attacker seeks to bypass.
- **LLM-based Meta-modelling For Rule Improvement:** We develop a meta-modelling approach leveraging Large Language Models (LLMs) to produce actionable rule improvement recommendations aimed at increasing *coverage* against evasions. Here, coverage refers to how well a detection rule successfully identifies the malicious behaviour it was designed to detect.
- **Comprehensive Evaluations:** We evaluate our approach on the 2025 Windows/PowerShell SIGMA rules, and release our code to ensure future repeatability.¹

The remainder of this paper is structured as follows. Section 2 compares this paper with related works, Sect. 3 presents the background knowledge, and Sect. 4 describes our threat model. Section 5 details our proposed system architecture, while Sect. 6 presents the experimental evaluation. Section 7 presents some key discussions and insights, while Sect. 8 concludes this paper.

¹ <https://github.com/HannesWid/thesis-artifacts>.

2 Related Works

The primary objective of this paper is to identify potential blind spots in an organisation’s cyber defence strategy arising from SIEM rules that attackers can evade. The secondary objective is to provide actionable recommendations to improve these evadable SIEM rules. While existing approaches in the literature address the primary objective, to the best of our knowledge, none address the secondary; consequently, a direct comparison is not possible. In this context, we compare our approach to numerous prior works on threat detection mechanisms, the strategies used to evade them, and methods for detecting such evasions in enterprise networks. Insights from these approaches guide our methodology to achieve the primary and secondary goals of this paper.

2.1 Threat Detection Mechanisms

Alsaheel et al. [27] presented a framework called ATLAS that constructed end-to-end attack stories from various security audit logs. They correlated individual attack instances to detect Advanced Persistent Threats (APTs), using a combination of causality analysis, natural language processing, and machine learning (ML) techniques to identify attack sequences. Schneider et al. [28] utilised semantic analysis to group similar findings from reports generated by different security tools. Their approach consolidated duplicate vulnerability reports, enabling security analysts to recognise instances of the same vulnerability across multiple reports, thereby reducing redundant analysis efforts.

NODOZE, developed by Hassan et al. [10], complemented existing threat detection systems by leveraging historical and contextual information to generate causal dependency graphs. Each edge in the graph was assigned an anomaly score based on the frequency of related events, and scores propagated along neighbouring edges to produce an aggregated score for alert triaging. The sub-graph with the highest anomaly score was extracted and considered the adversary’s presumed path. By combining SIEM data with attack goals, Sadlek et al. [20] generated attack paths representing potential cyber threat scenarios. Using SIEM alerts, evidence paths were constructed similarly, and a matching algorithm compared them to attack paths to enrich the process with additional information pertaining to severity levels, ATT&CK tactics, and asset criticality.

Although the above approaches perform well on their respective use cases, they do not consider the threats against the detection mechanisms themselves, arising from attackers deliberately evading them. These threats can compromise system integrity and reliability, potentially leading to safety hazards or security breaches depending on the attacker’s goals [16, 32, 33]. Moreover, when an approach [20] does provide enriched attack diagnostic information, this aspect alone does not directly translate to actionable recommendations for security analysts.

2.2 Evasion of Threat Detection Mechanisms

Evasion attacks are designed to bypass detection mechanisms while preserving the underlying malicious intent and functionality of the threat. Holm and Hyl-

lienmark [17] conducted an empirical study on anti-malware evasion techniques and evaluated their effectiveness against two antivirus tools. Across tens of thousands of tests spanning multiple evasion techniques, the authors reported that over 54% of the crafted evasions successfully bypassed anti-malware scans, with 5% even resulting in an interactive shell. They further demonstrated that payload compression, encryption schemes, and the use of common data formats were particularly effective. Notably, evasion attacks delivered through Windows Portable Executable (PE) files, Windows scripts executed via `cmd.exe` or `powershell.exe`, or data executed through trusted system binaries (e.g., `mshta.exe`, `wscript.exe`, or via macros embedded in Excel documents) achieved the highest success rates. Mezawa et al. [21] showed how evasion attacks could substantially degrade the performance of ML-based malicious PowerShell detection systems. Their approach modified malicious scripts by incorporating aspects of benign programs, thereby preserving the malicious functionality and successfully circumventing detection.

While the aforementioned approaches demonstrate how detection mechanisms in general can be bypassed, they do not focus on evasion techniques specific to SIEM rules, which present unique challenges. Techniques such as reordering, substitution, insertion, and omission produce semantically valid, mutated versions of the underlying malicious command while preserving its intended functionality. These mutations alter the command’s structure and reduce its syntactical similarity to the (closest) matching SIEM detection rules, increasing the likelihood of successful evasion. In our approach, this semantic-syntactic dissonance is leveraged to flag potential evasion attempts.

2.3 Identifying Evasions of Threat Detection Mechanisms

Numerous approaches in the literature focus on detecting evasion attempts against organisational cyber defences. In this context, we limit our comparison to evasion detection in operational settings most closely related to this work: Microsoft Windows PowerShell systems and Rule-based SIEM systems.

Microsoft Windows PowerShell Systems: Li et al. [19] develop a semantic-aware and de-obfuscation-based attack detection system capable of identifying malicious scripts that employ various obfuscation techniques to evade detection. Ryan Stamp [26] introduced a novel algorithm to detect Living-Off-The-Land (LOL) attacks where native binaries are abused to achieve malicious intent. Similarly, Ongun et al. [29] proposed an active learning framework called Living-Off-The-Land Detection with Active Learning (LOLAL), to detect LOL attacks using an ensemble classifier trained on vectorised embeddings of command-line representations. Finally, Vinay and Mangal [30] create the SCADE framework to identify adversarial patterns and detect LOL attacks through anomalous command-line activity. Although effective at detecting evasions through security events, these approaches do not provide organisations with recommendations to remediate defensive gaps caused by evadable SIEM rules.

Rule-Based SIEM Systems: SIEM systems help organisations identify and respond to potential security threats and vulnerabilities. As these systems are the focus of this paper, we deemed the work by Uetz et al. [22] to be the closest for comparison. In their work, the authors explored how SIEM rules pertaining to Windows Process Creation from the publicly available SIGMA rule set could be evaded. To address this, they proposed a proof-of-concept system called the Adaptive Misuse Detection System (AMIDES), which detects when SIEM rules are being evaded. Our work extends this approach by not only detecting rule evasions but also offering actionable recommendations to security analysts for improving the evadable SIEM rules. While AMIDES employs support vector machines for both misuse classification and rule attribution, our method leverages fine-tuned transformer models, which capture complex semantic relationships beyond simple lexical similarities in PowerShell and Windows commands [23].

3 Preliminaries

This section provides the background knowledge required to understand the concepts used in this paper. It covers rule-based SIEM systems built on the SIGMA detection format and the ELK stack, as well as the ML techniques used in our proposed approach.

3.1 Security Information and Event Management

A Security Information and Event Management system consists of two core components: Security Information Management (SIM) and Security Event Management (SEM). SIM focuses on the collection, storage, and analysis of historical log data to support investigations and retrospective analysis. In contrast, SEM provides real-time monitoring, event correlation, and alerts to identify and respond to ongoing security incidents. By aggregating and analysing logs across an organisation’s IT infrastructure [18], SIEM platforms enable the detection of anomalous behaviour and known threat signatures, while generating timely alerts for security teams [15].

The SIGMA Detection Format: SIGMA [8] is an open-source project that provides more than 3,000 generic SIEM rule specifications written in YAML (Yet Another Markup Language). These generic rules can be automatically translated² into formats supported by widely used SIEM platforms, including Splunk Enterprise Security, Microsoft Sentinel, and Elastic Security (using Elastic Query Language or EQL).

Structurally, a SIGMA rule is composed of several fields, with the *detection* component defining the behaviour the rule must flag. It consists of one or more *selections*, each representing a set of search terms or matching conditions. These

² <https://sigconverter.io/>.

```

detection :
  selection :
    Data | contains | all:
      - 'powershell'
      - 'nslookup'
      - '[1]'
    Data | contains:
      - '-q=txt http'
      - '-querytype=txt http'
  condition: selection

```

Fig. 1. SIGMA Detection Component Example

selections can be combined using logical operators such as AND and OR to express more complex detection logic. An example of this detection component of a SIGMA rule is shown in Fig. 1. Here, the rule triggers an alert if a log entry contains the strings `powershell`, `nslookup`, and `[1]`, along with at least one of `-q=txt http` or `-querytype=txt http` strings.

The ELK Stack: The ELK Stack³ consists of three core components: Elasticsearch, Logstash, and Kibana. Agents, which are daemon processes running on hosts, gather data in the form of log files and forward them to Logstash. Logstash provides a processing pipeline that deduplicates, aggregates, and transforms incoming data for ingestion into Elasticsearch. Once processed, the data is indexed and stored in Elasticsearch, which functions as a high-performance search engine. Finally, Kibana offers an intuitive interface to explore and analyse the indexed data, providing visual insights into the organisation’s security events.

3.2 Machine Learning Techniques

Since the approach used in this paper employs machine learning techniques in the form of transformer-based LLMs and ensemble learning with Random Forest regressors, we provide an overview of these approaches.

Transformers: The underlying mechanism behind LLMs such as OpenAI’s ChatGPT, or Anthropic’s Claude is the transformer architecture, which is essentially a deep neural network based on self-attention [11]. Transformers are specifically designed to process sequential data, such as natural language, where both order and context are crucial.

Random Forest Regressors: A Random Forest (RF) Regressor is an ensemble learning method used for regression tasks. It combines the predictions from several individual decision trees to produce a final output [7]. Normally, the prediction of an RF Regressor is the *average* of the predictions of all individual

³ <https://www.elastic.co/docs/get-started/the-stack>.

trees. However, in this paper, we use the *multi-output* variant of the RF Regressor, where instead of a single (scalar) output prediction, the model outputs a vector of predictions corresponding to multiple target variables. Each tree produces a vector of predictions, and the final output is obtained by averaging these vectors across all trees. Formally, if the ensemble consists of N trees, the output $\hat{y}$ for an input $\bar{x}$ is given by:

$$\hat{y} = \frac{1}{N} \sum_{t=1}^N h_t(\bar{x})$$

where $h_t(\bar{x})$ represents a vector of predictions from the t -th tree.

4 Threat Model

In this paper, we adopt the threat model from Biggio et al. [12], where the attacker is characterised by their knowledge, goals, strategies, and capabilities.

Attacker Knowledge: We assume a Grey-box level of knowledge where the attacker possesses partial knowledge of the SIEM environment, such as the vendor and the general types of logs collected (e.g., Windows Event Logs, Sysmon). However, they do not have access to specific detection rules or alert thresholds.

Attacker Goals - Since evasion attacks directly compromise the detection capabilities of SIEM systems, we assume the attacker’s primary goal to be integrity violations. In more advanced scenarios (out of scope for this paper), the attacker may also seek to cause availability violations by generating an incessantly high number of false alarms, causing alert fatigue [10], and reducing system usability.

Attacker Strategies - We assume that the attacker can employ the following strategies to evade detection while still maintaining the malicious intent and functionality of the underlying command or script:

- *Substitution*: Replacing a command or its parameters with an alias.
- *Reordering*: Changing the order of parameters within a command.
- *Omission*: Removing redundant characters or parameters.
- *Alternative Method*: Using a different command to achieve the same functionality.
- *Insertion*: Adding redundant characters, parameters, or arguments.

Attacker Capabilities - We assume the attacker has access to a Microsoft Windows PowerShell terminal within the organisational network, either through an endpoint or a network appliance. This access allows them to schedule and execute commands and scripts. In addition, they may observe SIEM alerts or system responses to infer aspects of the underlying detection mechanism, which then can be used to craft effective evasions.

5 Proposed System Architecture

Since the foundation of this paper rests on understanding how SIEM rules can be evaded, we conducted a structured analysis of all 215 Windows/PowerShell rules from the 2025 SIGMA repository, the most widely used corpus of open-source SIEM rules. We restricted our study to this subset due to the substantial manual effort that a comprehensive inspection of all 3,000 rules would entail, as well as the limited accessibility of certain log sources (e.g., from proprietary cloud services or commercial appliances) that many rules depend upon. Focusing on the Windows/PowerShell subset also ensured direct comparability with the state-of-the-art. By systematically applying the attacker strategies defined in our threat model in Sect. 4, mapping each evasion technique to its corresponding SIGMA rule and manually validating detection outcomes in the Elastic Security SIEM tool, we found that approximately 24% (or 51) of the analysed rules were potentially evadable, leaving large blind spots in organisational cyber defences. The architecture proposed in this section directly responds to our findings, aiming to harden SIEM systems against evasion attacks.

5.1 Overview

In conventional SIEM deployments, commands captured from execution logs, e.g., via Elastic agents, are matched against pre-defined SIEM rules to identify potential malicious activity. However, given the evasion vulnerabilities identified in our analysis, our proposed approach includes two modules: a SIEM Rule Evasion Detection module, which identifies evasive commands (that were not detected by the SIEM) and maps them to the corresponding SIEM rules they bypass, and a SIEM Rule Improvement module, which produces actionable recommendations for hardening those rules. We now proceed to describe these modules in detail.

5.2 SIEM Rule Evasion Detection Module

This module operates on commands that are not flagged as malicious by the SIEM. This includes both benign commands and potential false negatives, i.e., malicious commands that evade SIEM detection. The module uses a fine-tuned transformer to generate embeddings from the tokenised versions of these commands and compares them against the pre-processed embeddings of the SIEM rule branches. It outputs the flagged evasive commands along with the top- k most likely evaded SIEM rules as shown in Fig. 2. To achieve this, the module measures the cosine similarity between the command embedding and the embeddings of all the SIEM rule branches. The SIEM rule branch embeddings are then ranked based on their similarity scores, and if the maximum score exceeds a pre-defined threshold, the command is flagged as evasive, with the top- k SIEM rules returned. By leveraging transformer-generated embeddings, this approach captures the semantic intent of both commands and rules, enabling detections that traditional rule-matching methods would almost certainly miss.

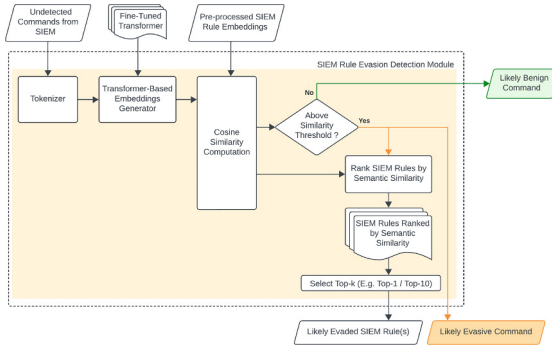


Fig. 2. SIEM Rule Evasion Detection Module

The SIEM Rule pre-processing pipeline consists of two key steps: pruning and feature extraction. Pruning removes environment-specific components, such as hardcoded hostnames, that are irrelevant to the core detection logic, while feature extraction isolates the search terms associated with each rule branch. The fine-tuned transformer in our approach is essentially CmdCaliper [23], enriched with specialised knowledge from our dataset of 125 entries consisting of aliases, small rule examples, and corresponding evasive commands, which we make publicly available alongside our code. The model was fine-tuned using CosineSimilarityLoss for 5 epochs with a batch size of 16. An example of the fine-tuning data format is presented in Fig. 3. This fine-tuned transformer then generates the embeddings for the commands and the rule branches.

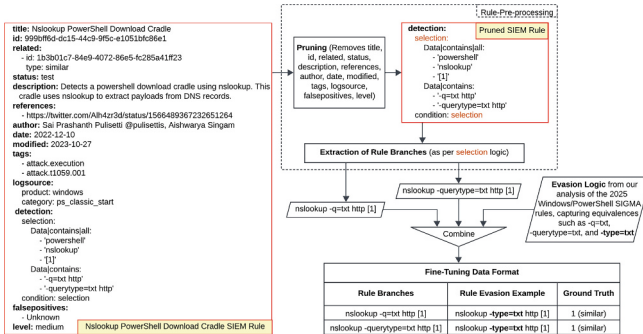


Fig. 3. Fine-Tuning Data Format Example

As mentioned earlier, during our analysis of the 2025 Windows/PowerShell SIGMA rules, we found that almost a quarter of the rules could be evaded. Figure 3 illustrates this using the NSLookup PowerShell download cradle rule as an example, which can be evaded by substituting `-q=txt` or `-querytype=txt` with

`-type=txt`; this issue was also reported to the SIGMA community maintainers, who subsequently updated the rule.⁴ In the figure, we first prune the rule to retain only the detection section. We then identify the relevant rule branches according to the selection logic and define the binary ground-truth similarity labels for their corresponding evasive variants. These labels indicate whether the two entries are semantically similar (1) or not similar (0). Since `-q=txt`, `-querytype=txt`, and `-type=txt` are equivalent, the rule-evasion pairs `nslookup -q=txt http [1]` and `nslookup -type=txt http [1]`, as well as `nslookup -querytype=txt http [1]` and `nslookup -type=txt http [1]`, were assigned a label of semantically similar (1).

5.3 SIEM Rule Improvement Module

In this paper, rule improvement refers to the process of updating a SIEM rule to more comprehensively detect malicious behaviour and identify evasions that the original rule failed to catch. To the best of our knowledge, this work represents the first approach towards enhancing the coverage of SIEM rules using LLMs. To achieve this goal, we introduce the SIEM Rule Improvement Module, which takes as input an evasive command and its corresponding evaded SIEM rule, and returns a rule improvement recommendation. It does so by routing a generated prompt to the most suitable LLM from a pool comprising Codestral-2501 (Mistral), GPT-4o (OpenAI), and Gemini-2.0-Flash (Google), all selected solely due to their free, generous usage limits. The optimal LLM is chosen by a random-forest regressor that assigns each candidate a *suitability score* (T) based on three factors: *effectiveness* (E), which measures whether the updated rule successfully detects the evasion; *retention* (R), which assesses how much of the original rule structure is preserved so as not to alter its intended behaviour; and *specificity* (S), which evaluates whether the updated rule is overly narrow or broad and its propensity to generate false positives. The highest-scoring LLM then receives the prompt via an API call, as illustrated in Fig. 4.

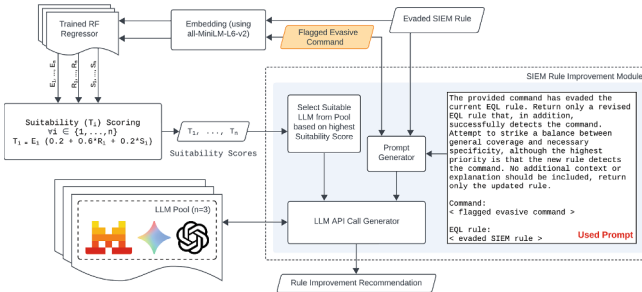


Fig. 4. SIEM Rule Improvement Module

⁴ <https://github.com/SigmaHQ/sigma/commit/54496e2>.

To train the random forest regressor (see Fig. 5), we use a dataset consisting of approximately 350 rule-evasion pairs (which we also make publicly available alongside our code). Each rule-evasion pair is submitted to all three LLMs, and their responses are manually evaluated to produce ground-truth triplets $(E_1, \dots, E_n), (R_1, \dots, R_n),$ and $(S_1, \dots, S_n)$, where $n = \text{number of LLMs in the pool}$. Since the random forest cannot directly process text, both the SIEM rule and the evasive command are first embedded into numerical vectors using the all-MiniLM-L6-v2⁵ sentence transformer. These vectors serve as input features, while the aforementioned triplets form the corresponding multi-output regression targets. The model is thus trained to predict, for any given rule-evasion pair, how suitable each LLM is expected to be at generating a useful rule improvement, allowing the system to route each query to the most appropriate LLM automatically.

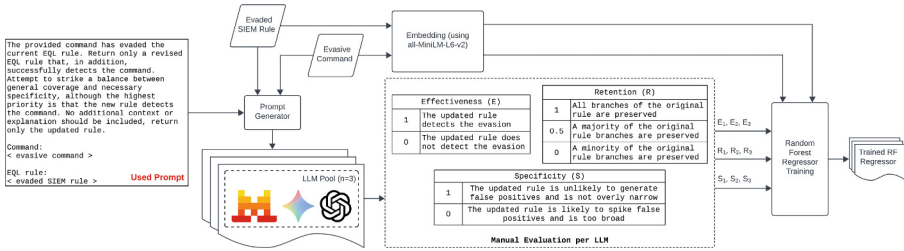


Fig. 5. Training the Random Forest Regressor

The formula for $T = E(0.2 + 0.6R + 0.2S)$ in Fig. 4 weights effectiveness most heavily because if the updated rule fails to detect the evasion, the improvement is fundamentally worthless. Retention is weighted above specificity because we consider preserving the original rule’s coverage more important than the risk of introducing additional false positives. Also, any false positives introduced by the rule update can generally be tuned away more easily, meaning that a low specificity can be managed. Finally, while we acknowledge that LLM selection is subject to several user-defined criteria (ethical, legal, cost, and sustainability considerations), in our limited trial of the selected range of free LLMs, we found that GPT-4o outperformed both Gemini-2.0-Flash and Codestral-2501 across all our suitability factors (see Appendix B for measures).

5.4 Workflow

For ease of understanding, our approach consists of two phases: pre-deployment and post-deployment. During the pre-deployment phase, system designers create the fine-tuned transformer and the pre-processed embeddings of SIEM rule

⁵ <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.

branches required for the SIEM Rule Evasion Detection module. They also create the LLM pool, which is instantiated *once* with a fixed number of LLMs depending on the organisation’s available budget, and train the random forest regressor needed for the SIEM Rule Improvement module. Additionally, they configure the evasion detection pipeline used in the post-deployment phase, as illustrated in Fig. 6.

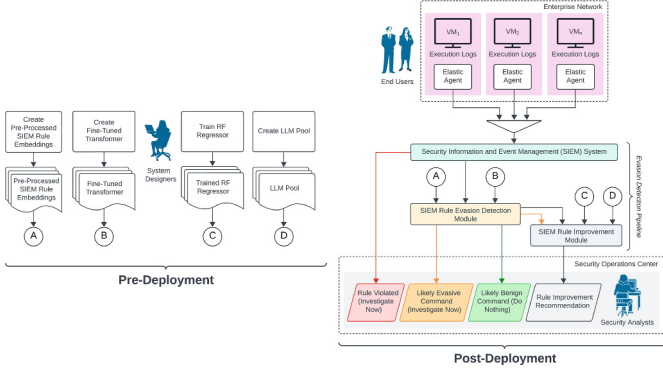


Fig. 6. Workflow of our Proposed Approach

In the post-deployment phase, execution logs from multiple virtual machines (VMs) across the enterprise network are collected via Elastic agents and forwarded to the SIEM, which monitors for rule violations and generates alerts for the Security Operations Center (SOC). Commands that go undetected by the SIEM are routed into our evasion detection pipeline, where the SIEM Rule Evasion Detector postulates them as potential evasions or benign activity. If an evasion attempt is identified, the corresponding command and SIEM rule are passed to the SIEM Rule Improvement module, which generates actionable recommendations for security analysts in the SOC.

6 Experimental Evaluation

In this section, we evaluate our approach to the tasks of detecting SIEM rule evasions and generating rule improvements. We first describe the experimental setup, which provides the necessary context for interpreting the reported results.

6.1 Dataset Creation and Experimental Setup

To evaluate our SIEM Rule Evasion Detection Module, the creation of a dataset was necessary. This is because datasets in the security operations domain are typically kept private due to the sensitive information they contain, and the closest existing resource - the evasions presented by Uetz et al. [22], dated back

to 2021 with no publicly available benign data, making direct reuse impossible. To this end, we created a dataset of 500 evasive commands and 4,500 benign commands. The benign commands were derived from a publicly available PowerShell malware detection dataset.⁶ After removing the malicious scripts, the Mistral 7B LLM was prompted to extract cmdlets and their usage context from each benign script - a straightforward information retrieval task well-suited to a smaller, locally-runnable model. From these, an initial set of 180 randomly selected cmdlets were used to craft commands. These commands were then manually verified against the Microsoft PowerShell documentation and subsequently mutated using OpenAI's GPT-4o (a larger model trained on broader data with significantly more parameters, making it better suited for complex generative tasks) resulting in a final total of 4,500 benign commands.

The 500 evasive commands were based on the 51 SIEM rules we identified at the beginning of Sect. 5 as potentially evadable, and were crafted using the evasion techniques from Sect. 4 to which these rules were susceptible. We then executed our dataset of evasive and benign commands on a virtual machine. The resulting execution logs were collected via Elastic Agents, processed through the Elastic Security SIEM tool, and analysed by our SIEM Evasion Detection pipeline. The similarity threshold (0.77) was selected empirically from the ROC curve as the operating point that provided the desired trade-off between evasion detection accuracy and the false positive rate.

Out of the same 500 evasive commands, 350 were used to train the RF regressor (in our SIEM Rule Improvement Module), leaving 150 commands for evaluating the rule improvement process. The evaluation was conducted in two ways: first, by incorporating the rule improvement recommendations generated by our approach into the SIEM system and measuring the new evasion detection accuracy; and second, through expert validation. Upon receiving an updated rule from the most suitable LLM, a triplet was constructed comprising the original rule, the evasive command, and the updated rule.

Four security analysts at a cybersecurity company with extensive experience in SIEM operations independently evaluated the same set of 150 triplets. Using their expertise, they qualitatively assessed each updated rule and categorised it as effective, helpful, or flawed. A rule was considered effective if it could be deployed with minor or no modifications; helpful if it was not directly deployable but still provided actionable insights for improving the original rule; and flawed if it offered little to no value in enhancing coverage against evasions. Expert review was particularly important because some rule improvement recommendations removed rule branches or made them overly specific, requiring a deeper assessment beyond simple accuracy measurements.

6.2 SIEM Rule Evasion Detection and Attribution

We begin by stating an important caveat - a direct comparison between our approach and AMIDES was not possible, as their set of benign commands was

⁶ https://huggingface.co/datasets/rr4433/powershell_malware_detection_dataset.

unavailable due to security and privacy restrictions of their partner organisation. It is also important to note that AMIDES was built for a simplified subset of the 2021–22 SIGMA rules, whereas our approach is based on the 2025 specification, which introduced structural changes that AMIDES was not designed to handle. Due to these differences, we could not use a common dataset for comparison without reverting to the 2021–22 SIGMA rules, which would have been counter-productive, as doing so would ignore the improvements, fixes, and more realistic, production-grade detection complexities present in the 2025 ruleset. Nevertheless, the comparison remains informative, as both approaches address a similar task under broadly comparable conditions.

Table 2. Performance comparison with the state-of-the-art, AMIDES [22]

SIEM Rule Evasion Detection		
System	Accuracy (Score)	False Positive (Score)
AMIDES [22]	0.70	0.0000
Our Approach	0.71	0.0093
Evasion Rule Attribution Accuracy		
System	Top-1 (%)	Top-10 (%)
AMIDES [22]	63%	95%
Our Approach	98%	99.2%

Relating to the results in Table 2, we remark that on a similar task used to evaluate AMIDES, our system detected 71% of evasions while maintaining a low false positive score of 0.0093. This indicates that our approach achieves comparable (and slightly better) evasion detection accuracy in relation to AMIDES. Furthermore, our system outperforms AMIDES on the rule attribution task in both the Top-1 and Top-10 scenarios, which measure the percentage of evasions for which the correct underlying rule is ranked first or appears within the top ten predicted rules, respectively.

It is also noteworthy that while AMIDES reports a false positive score of zero, it was evaluated on a dataset of over 155,373,027 benign events, compared to our 4,500 benign commands. The much larger evaluation set in AMIDES’ case likely contributed to the reported false positive score of zero, whereas the smaller size of our benign dataset makes even a few false positives more noticeable. As an example, 100 false positive detections on our dataset would yield a false positive score of 0.02, whereas the same number for AMIDES’s dataset would correspond to a score of 6×10^{-7} , or simply, zero.

6.3 SIEM Rule Improvements

From Sect. 6.1, the rule improvement process was adjudicated using two methods:

SIEM Integration: After integrating our recommended rule updates in the SIEM, we executed the evasive commands on a VM. This resulted in 105 alerts, meaning 70% of the 150 recommendations successfully detected their corresponding evasions.

Expert Validation: When presented with the rule improvement recommendations generated by our system, the security analysts assessed 46 out of 150 recommendations as effective. An additional 71 recommendations were considered helpful, and the remaining 33 recommendations were deemed flawed or not helpful.

7 Discussions

In this section, we present a critical analysis of our approach, examining its practical applicability and associated limitations.

7.1 Towards Practical Applicability

While fine-tuned transformers proved useful for detecting SIEM rule evasions, it is important to recognise the fundamental limitations of the LLMs used for rule improvements. LLMs are largely non-factual systems that are prone to hallucinations and should not be blindly trusted [9]. They do not possess a semantic understanding of the language they process or the (human-aligned) meaning of individual tokens; rather, they are trained to perform next-token prediction, statistically estimating the most probable output given the preceding context. This limitation is captured by the *stochastic parrots* metaphor [13], which highlights an LLM’s capacity to generate text that appears coherent and plausible yet may be factually incorrect. In this context, relying solely on an LLM to autonomously update SIEM detection rule logic is not advisable. Although LLMs can assist in generating actionable recommendations for rule improvements, the review, validation, and implementation of any modifications must always remain under the purview of a qualified security analyst.

Furthermore, because the use of machine learning techniques in cybersecurity is generally considered to be a double-edged sword, attackers may actively attempt to evade our transformer-based detection mechanism itself. In theory, this could occur in two ways. First, an adversary could seek to poison the dataset used to fine-tune the transformer, with the objective of causing the model to disregard context-specific features that are ordinarily leveraged to detect evasive behaviour at runtime. We consider this scenario largely impractical given the significant engineering overhead and computational cost associated with iterative retraining of large transformer models. Moreover, there is no guarantee that such poisoning would reliably induce the model to suppress or override the desired context-specific features. In addition, this process would likely produce anomalous spikes in the transformer’s training loss curves [4], producing a detectable signal that could be leveraged to identify and mitigate the poisoning attack. Alternatively, an attacker could inject irrelevant or junk tokens

into executed commands to deliberately reduce the cosine similarity between the command and its (closest) matching SIEM rules. However, this can be mitigated by pre-processing steps that prune redundant or non-informative tokens prior to the analysis, or by employing error-correcting output codes as demonstrated in earlier works [2,3].

7.2 Limitations

The current approach does not explicitly account for the use of negations within the branching logic of the SIEM rules. Since the transformer is employed solely to generate embeddings for rules and commands, the presence of a negation generally has a relatively small impact on the resulting cosine similarity measurement process. This occurs because the embeddings capture the overall semantic context of a rule, which can diminish the effect of individual negations. In practice, however, this limitation may lead to a rise in false positives when a negation substantially changes the intended meaning of a rule. This limitation can be addressed through additional pre-processing to explicitly identify and handle negations, or by fully re-training the transformer to recognise and account for negation clauses.

Another limitation of our method lies in the dimensionality of the embeddings. The current approach requires embeddings of rules and commands to have identical dimensions, which may not naturally occur. When the embeddings differ between a rule and a command, corrective measures normally include padding the shorter vector with zeros or projecting both vectors into a lower-dimensional space. However, both strategies potentially reduce the accuracy of the cosine similarity process and the resulting evasion detection. This limitation can likely be addressed through fixed-size vector embedding matrices as demonstrated in earlier works [6].

8 Conclusion and Future Works

This paper addresses the task of detecting the evasion of SIEM rules, a threat that can disrupt operations and weaken an organisation’s security posture. We began by analysing the Windows/PowerShell SIGMA detection rules from 2025, finding that out of 215 rules, 45 could be fully evaded and 6 were partially evadable. To address this issue, we proposed a fine-tuned transformer model to detect evasion attempts by identifying the semantic-syntactic dissonances between incoming commands and their closest matching SIEM rules. Our method was able to detect evasion attempts targeting these rules with an accuracy of 71% while maintaining a low false positive rate of less than 1%, demonstrating comparable performance to the current state-of-the-art method, AMIDES, while requiring a fraction of the training data and the associated training effort. In addition, our approach could generate rule improvement recommendations based on a given evasion and its corresponding SIEM rule. Furthermore, when performing a similar task, our method outperformed AMIDES in identifying

the specific evaded rule, achieving an accuracy gain of up to 35% in both Top-1 and Top-10 rule scenarios.

When evaluating the rule improvement recommendations, security analysts with expert knowledge on rule adaptations found that 46 out of our 150 recommendations were fully effective and could replace the original rules with only minor adjustments, 71 were considered helpful, and 33 were flawed. Using our generated recommendations, we also provided fixes to the SIGMA repository maintainers, resulting in several rules being corrected. Overall, this work demonstrated the practical value of transformers and LLMs for both detecting SIEM rule evasions and generating improved rules, contributing directly to closing the blind spots in an organisation’s cyber defence strategy.

Regarding future works, the current process of rule adaptation and augmentation remains painstakingly slow, creating substantial security gaps that adversaries could exploit over extended time periods. While the approach proposed in this paper aims to accelerate this process, further investigation is needed to critically evaluate the timing performance of evasion detection and to optimise bottlenecks within this pipeline. In this context, this paper lays the groundwork for such a study by establishing a proof of concept that demonstrates the viability and usefulness of our approach - a necessary first step before systematic performance optimisations can be meaningfully pursued.

Beyond performance optimisations, another important direction concerns the generalisability of our approach. While effective for Windows/PowerShell, its effectiveness in other environments, such as Linux systems, cloud platforms, or web applications, remains unexplored. Further research is therefore required to assess its applicability across the diverse range of SIEM rule types deployed in enterprise environments.

In addition to generalisability, the practical deployment of our proposed approach requires assessing execution time and resource overhead. Since SOC’s are already overwhelmed by large volumes of SIEM logs each day, any supplementary system for detecting rule evasions and suggesting improvements should operate efficiently. However, given that evasion detection inherently requires human supervision to review and act on the findings, the human operator remains the practical bottleneck rather than computational performance. As such, investments in security competence and human oversight are key considerations for the future deployment of such systems.

Another practical direction is to evaluate whether the proposed approach exhibits performance differences across SIEM platforms beyond ELK. Since SIEM systems may differ in their rule formats and operational workflows, a comparative study across existing SIEM solutions would help assess the broader applicability of our method.

Finally, the detection of rule evasions in our approach focused primarily on single commands extracted from agent logs. In practice, however, sophisticated intrusions rarely unfold in a single step - they typically involve a coordinated sequence of actions, each potentially evading a different rule. In this context, extending our approach to analyse sequences of commands within a defined

temporal window could enable the detection of complex, multi-step attacks that cannot be reliably identified through single-command analysis alone, and is a line of research worth pursuing.

Acknowledgments. This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. We would also like to express our sincere gratitude to the security analysts at Sectra Communications, Sweden, for their invaluable support in this work.

A Appendix

B LLM Suitability Scores

For the rule improvement module, we evaluated the suitability of each LLM-generated rule update according to three criteria: effectiveness, retention, and specificity. The criterion-level suitability scores for each evaluated LLM are reported in Table 3 and correspond to the average across the 350 training inputs per model.

Table 3. LLM suitability criterion scores

Suitability Criteria			
LLM	Effectiveness	Retention	Specificity
Gemini-2.0-Flash	0.89	0.35	0.37
GPT-4o	0.98	0.50	0.65
Codestral-2501	0.80	0.18	0.15

References

1. Alahmadi, B.A., Axon, L., Martinovic, I.: 99% false positives: a qualitative study of SOC analysts’ perspectives on security alarms. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 2783–2800 (2022)
2. Verma, G., Swami, A.: Error correcting output codes improve probability estimation and adversarial robustness of deep neural networks. In: Advances in Neural Information Processing Systems, p. 32 (2019)
3. Song, Y., Kang, Q., Tay, W.P.: Error-correcting output codes with ensemble diversity for robust learning in neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, no. 11, pp. 9722–9729 (2021)
4. Gao, G., et al.: Try to poison my deep learning data? nowhere to hide your trajectory spectrum!. In: 2025 Network and Distributed System Security (NDSS) Symposium. Internet Society (2025)

5. Virvilis, N., Gritzalis, D.: The big four-what we did wrong in advanced persistent threat detection?. In: 2013 International Conference on Availability, Reliability, and Security, pp. 248–254. IEEE (2013)
6. Lin, Z., et al.: A Structured Self-attentive Sentence Embedding. arXiv [Cs.CL] (2017). <http://arxiv.org/abs/1703.03130>
7. Breiman, L.: Random forests. *Mach. Learn.* **45**(1), 5–32 (2001)
8. SigmaHQ. SigmaHQ/sigma: Main SIGMA rule repository. GitHub (2025). <https://github.com/SigmaHQ/sigma/>
9. Kucharavy, A.: Fundamental limitations of generative LLMs. In: Large Language Models in Cybersecurity: Threats, Exposure and Mitigation, pp. 55–64. Cham, Springer Nature Switzerland (2024)
10. Hassan, W.U., et al.: Nodotze: combatting threat alert fatigue with automated provenance triage. In: the Network and Distributed Systems Security Symposium (2019)
11. Vaswani, A., et al.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30** (2017)
12. Biggio, B., et al.: Security evaluation of support vector machines in adversarial environments. In: Support Vector Machines Applications, pp. 105–153. Springer International Publishing, Cham (2014)
13. Bender, E. M., Gebru, T., McMillan-Major, A., Shmitchell, S.: On the dangers of stochastic parrots: can language models be too big?. In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (2021)
14. Sommer, R., Paxson, V.: Outside the closed world: on using machine learning for network intrusion detection. In: 2010 IEEE Symposium on Security and Privacy, pp. 305–316. IEEE (2010)
15. Yen, T.F., et al.: Beehive: large-scale log analysis for detecting suspicious activity in enterprise networks. In: Proceedings of the 29th Annual Computer Security Applications Conference, pp. 199–208 (2013)
16. Bergadano, F., Gupta, S., Crispo, B.: Techniques and metrics for evasion attack mitigation. *Comput. Secur.* 104802 (2025)
17. Holm, H., Hyllienmark, E.: Hide my payload: an empirical study of antimalware evasion tools. In: 2023 IEEE International Conference on Big Data (BigData), pp. 2989–2998. IEEE (2023)
18. Bhatt, S., Manadhata, P.K., Zomlot, L.: The operational role of security information and event management systems. *IEEE Secur. Priv.* **12**(5) (2014)
19. Li, Z., Chen, Q.A., Xiong, C., Chen, Y., Zhu, T., Yang, H.: Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pp. 1831–1847 (2019)
20. Sadlek, L., Yamin, M.M., Čeleda, P., Katt, B.: Severity-based triage of cybersecurity incidents using kill chain attack graphs. *J. Inf. Secur. Appl.* **89**, 103956 (2025)
21. Mezawa, Y., Mimura, M.: Evaluating the possibility of evasion attacks to machine learning-based models for malicious PowerShell detection. In: International Conference on Information Security Practice and Experience, pp. 252–267. Springer International Publishing, Cham (2022)
22. Uetz, R., Herzog, M., Hackländer, L., Schwarz, S., Henze, M.: You cannot escape me: detecting evasions of SIEM rules in enterprise networks. In: 33rd USENIX Security Symposium (USENIX Security 24), pp. 5179–5196 (2024)

23. Huang, S.Y., Yang, C.L., Lin, C.Y., Huang, C.Y.: Cmdcaliper: a semantic-aware command-line embedding model and dataset for security research. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 20188–20206 (2024)
24. Heath, B., Timmons, H., Cooney, P.: Solarwinds hack was 'largest and most sophisticated attack' ever (2021). <https://reut.rs/3b5ojqL>
25. Milmo, D., Kollewe, J., Ibrahim, M., Taylor, J., Quinn, B.: Slow recovery from IT outage begins as experts warn of future risks (2024). <https://www.theguardian.com/australia-news/article/2024/jul/19/microsoft-windows-pcs-outage-blue-screen-of-death>
26. Stamp, R.: Living-off-the-land abuse detection using natural language processing and supervised learning. [arXiv:2208.12836](https://arxiv.org/abs/2208.12836) (2022)
27. Alsaheel, A., et al.: ATLAS: a sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 3005–3022 (2021)
28. Schneider, P., Voggenreiter, M., Gulraiz, A., Matthes, F.: Semantic similarity-based clustering of findings from security testing tools. In: Proceedings of the 5th International Conference on Natural Language and Speech Processing, pp. 134–143. ICNLSP 2022 (2022)
29. Ongun, T., et al.: Living-off-the-land command detection using active learning. In: Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defences, pp. 442–455 (2021)
30. Vinay, V., Mangal, A.: SCADE: scalable framework for anomaly detection in high-performance systems (2024). [arXiv:2412.04259](https://arxiv.org/abs/2412.04259)
31. Cleghorn, L.: Network defense methodology: a comparison of defense in depth and defense in breadth. *J. Inf. Secur.* 4(3), 144 (2013)
32. Colaco, V.O., Nadjm-Tehrani, S.: Fast evasion detection & alert management in tree-ensemble-based intrusion detection systems. In: 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI), pp. 404–412. IEEE (2024)
33. Colaco, V.O., Nadjm-Tehrani, S.: Real-time evasion detection in tree ensemble automotive intrusion detection systems. In: 2025 IEEE Vehicular Networking Conference (VNC), pp. 1–8. IEEE (2025)